

O HackTown é um espaço apartidário

⚠ Qualquer manifestação político-partidária identificada antes, durante ou após a apresentação pode resultar na interrupção imediata da atividade e na exclusão do speaker da programação, sem prejuízo das medidas previstas no Termo de Speaker assinado.



HACK10WN
anos

**Como decisões de arquitetura
de software viram decisões
de custo na Cloud**

Felipe KiKo

A fatura chegou...

Aquele momento em que você abre o Cost Explorer numa segunda-feira

Quem é o KiKo?

- **Solution Architect @ Avantpro**
- +20 anos em Tech
- **AWS Community Builder**
- Leader do **AWS User Group Campinas**

Movido a Curiosidade, Pokémon, Café e
faturas baixas!



O meme que todo Dev conhece



EXPECTATIVA

- "vai dar bom no deploy"
- "é só um teste"
- "depois eu desligo"



REALIDADE

- \$4,327.82
- "rodando há 6 meses"
- *narrator: he never did*

Ninguém acorda e decide gastar \$50k em NAT Gateway!

Mas alguém...em algum momento, fez um `terraform apply` e nunca mais revisou...

O vilão não é a AWS...

As vezes é a Arquitetura!

PARECE OK...

- "Vamos logar tudo por segurança"
- "Coloca Multi-AZ, vai que..."
- "Deixa o autoscaling sem limite"
- "Mas a API está 100% funcionando..."
- "Minha aplicação tem ORM"

...MAS O CUSTO É OUTRO

- \$3k/mês em CloudWatch
- 2x custo em ambiente de Dev
- Fatura surpresa de \$800 em 4h
- Com payload de 500k!
- Sem cache e sem batch!

Má arquitetura de software = Má fatura

① Chatty microservices

50 chamadas HTTP entre serviços para 1 response = latência + data transfer + compute

② N+1 queries no banco

ORM que faz 1 query por item. 100 usuários = 101 queries = RCUs/WCUs explodindo

③ Monolito deployado como 10 Lambdas iguais

Mesmo código em todas, cold start de 3s, 512MB cada. Uma ECS task resolveria

N ...

A tese central desse nosso bate-papo

"Cada linha de código é uma linha na fatura."

Decisão
técnica



Decisão
financeira

Não é sobre cortar custo. É sobre **decidir com consciência.**

Compute

Onde tudo começa (e onde muito dinheiro morre)

Lambda vs EC2 vs ECS...contexto é tudo!

CENÁRIO	MELHOR OPÇÃO	PIOR OPÇÃO
Evento esporádico, <1s	Lambda	EC2 24/7
Carga constante, previsível	EC2/ECS	Lambda
Picos imprevisíveis	Lambda + ECS	EC2 fixo
Execução longa (>15min)	ECS/EC2	Lambda

Não existe bala de prata. Existe **contexto**.

Cenário: O Lambda disfarçado de EC2

O SETUP

- Função Lambda: 512MB RAM
- Execução: 14 minutos
- 10.000 invocações/dia
- Processamento de imagens

O CUSTO

\$2,800
por mês em Lambda 💀

Mesmo workload num ECS Fargate com Spot: ~\$180/mês.

É como pedir Uber pro trabalho todo dia quando um busão faria o mesmo trajeto!

Right-sizing: a instância que roda a 8%

ANTES

- `t3.xlarge` (4 vCPU, 16GB)
- CPU média: 8%
- RAM média: 12%
- "Mas e se precisar?"

DEPOIS

- `t3.small` (2 vCPU, 2GB)
- Economia: ~70%
- Autoscaling pra picos
- AWS Compute Optimizer recomendou

Dica: AWS Compute Optimizer é grátis!

Ele olha 14 dias de métricas e te diz o tamanho certo.

Spot + Graviton = combo de economia

SPOT INSTANCES

- Até **90% mais barato** que On-Demand
- Perfeito pra: CI/CD, batch, tolerant workloads
- "A instância que morre do nada"...não

GRAVITON (ARM)

- Mesma carga = **-20% custo** + mais performance
- Maioria dos workloads roda sem mudar código
- EC2, ECS, Lambda, RDS...todas!

Combo máximo: Right-size + Spot + Graviton = economia de 70-90% vs o "t3.xlarge on-demand padrão".

Resumo: Compute

DECISÃO	IMPACTO
Lambda pra tudo	● Pode explodir em execuções longas
EC2 sem right-size	● Dinheiro queimado todo mês
Ignorar Spot	● Perdendo até 90% de desconto
Ignorar Graviton	● -20% fácil sem esforço
Escolher compute por contexto	● Eficiência real

Banco de Dados

Onde os dados moram (e o dinheiro some)

É triste...mas usar de maneira errada

O CENÁRIO

- Tá lento? Adiciona um index, não...aumenta a máquina!
- SELECT, UPDATE, SELECT, UPDATE...
- Queries *mal criadas!*
- Bora guardar tudo, e esquecer depois
- Relatório em produção?!? 🤔

O RESULTADO...

Meleca!

DBA não trabalha sozinho, depende do desenvolvedor também, boa parte do tempo!

RDS Multi-AZ em Dev

O CENÁRIO

- "É pra simular produção"
- Ninguém faz failover test
- Ninguém precisa de HA em Dev

O RESULTADO

2x

custo sem necessidade

Multi-AZ é pra Produção, e olha lá! Qualquer coisa diferente é Single-AZ + Snapshot!

Aurora Serverless v2 vs Provisionado

PADRÃO DE USO	MELHOR OPÇÃO
Carga constante 24/7	Provisionado
Picos + períodos idle	Serverless v2
Dev/Staging	Serverless v2 (pausa = \$0!)

O segredo: Aurora Serverless v2 escala em ACUs. Se o mínimo for alto demais, você perde a vantagem. Configure `min_capacity` baixo.

DynamoDB: o perigo do Scan

```
// "Só vou buscar uns itens..."  
const result = await dynamodb.scan({ TableName: 'users' })  
// Tabela: 50GB  
// RCUs consumidas: MUITAS!  
// Fatura: CHORANDO!
```

ON-DEMAND

- Tráfego imprevisível
- Projetos novos
- Spikes desconhecidos

PROVISIONED + AS

- Padrões conhecidos
- Mais barato a longo prazo
- Previsibilidade na fatura

Cache que ninguém invalida

O CENÁRIO

- ElastiCache Redis `r6g.xlarge`
- 80% das keys expiradas/inúteis
- TTL: "não definimos" 🧑
- Hit rate: 12%

O CUSTO

\$400

por mês pra guardar lixo 🗑️

É como pagar aluguel de um depósito cheio de caixas vazias...

Defina TTL, monitore hit rate, ajuste o tamanho

Resumo: Banco de Dados

DECISÃO	IMPACTO
Mal desenvolvimento	● Infra e DBA choram
Multi-AZ em Dev	● 2x custo sem necessidade
Scan full table no DynamoDB	● Custo imprevisível
Cache sem TTL	● Dinheiro parado
Aurora Serverless pra Dev	● Pausa = \$0
Provisioned + auto-scaling	● Previsibilidade

Storage & Transfer

O custo que cresce silenciosamente

S3: o arquivo que fica no Standard eternamente

O CENÁRIO

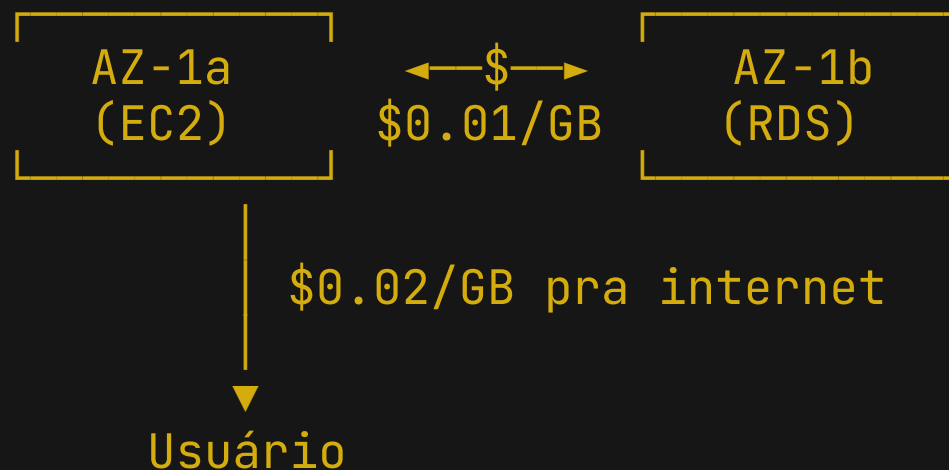
- Bucket de logs: 12TB
- Último acesso: 8 meses atrás
- Storage class: **S3 Standard**

COMPARAÇÃO DE CUSTO

CLASSE	CUSTO
Standard	~\$280
Glacier Deep Archive	~\$12
Economia	95%

Lifecycle Rules fazem isso automaticamente. Configure uma vez, economize pra sempre!

Data Transfer: o custo escondido



Microserviços conversando entre AZs = cada chamada tem "pedágio"

Coloque serviços que conversam muito na **mesma AZ** (com trade-off de resiliência)

NAT Gateway: O pedágio mais caro da VPC

\$0.045

por hora
(~\$32/mês só de existir)

\$0.045

por GB
processado

~\$119

3 AZs + 500GB
por mês 💀

Alternativas: VPC Endpoints (\$0.01/GB), Gateway Endpoints (S3/DynamoDB = grátis!),
ou revisar se precisa mesmo de NAT

Resumo: Storage & Transfer

DECISÃO	IMPACTO
S3 Standard forever	● 90%+ de desperdício
Sem VPC Endpoints	● NAT cobrando por tudo
Multi-AZ data transfer	● Pedágio silencioso
Lifecycle Rules	● Automação de economia
Gateway Endpoints	● S3/DynamoDB grátis

Observabilidade & Deploy

Quando "ver tudo" custa mais que o problema

CloudWatch Logs: o *logzinho* de \$3k/mês

```
// O código inocente que destrói orçamentos:  
exports.handler = async (event) => {  
  console.log(JSON.stringify(event)) // 🐱 CADA invocação  
  // ... lógica real  
}
```

OS NÚMEROS

- 50M invocações/mês
- ~2KB por invocação
- Ingestão: **100GB/mês**
- Sem retention policy: **crece infinito**

A SOLUÇÃO

- Log levels (ERROR, WARN em prod)
- Retention policy (7-30 dias)
- Sampling em high-volume
- Structured logging (filtra melhor)

Métricas customizadas: cada métrica = dinheiro

O PREÇO

- \$0.30/métrica/mês (primeiras 10k)
- Parece pouco...
 - Até ter 500 métricas × 5 dimensões
 - E nunca serem usadas

O TESTE

Pergunte pra cada métrica:

"Alguém acorda de madrugada por causa dessa métrica?"

Se não → **DELETE**

Ambientes zumbis vs efêmeros

AMBIENTE	STATUS	CUSTO/MÊS
staging - v1	"não mexe"	\$340
dev - kiko	"tô usando" (há 4 meses)	\$180
test - feature - x	PR já merged	\$95
demo - cliente	"semana que vem"	\$220
Total de zumbis		\$835/mês

Ephemeral environments com TTL automático. Morreu o PR, morre o ambiente. Sem humano no loop.

CI/CD: o build PESADO!

ANTES

- CodeBuild: 7GB RAM
- Build: **40 minutos**
- 30 builds/dia
- `BUILD_GENERAL1_LARGE`
- ~\$400/mês

DEPOIS

- Cache de layers Docker
- Testes paralelos
- `BUILD_GENERAL1_MEDIUM`
- Build: **12 minutos**
- ~\$90/mês

Economia de 77% com otimizações que melhoram o DX e o custo ao mesmo tempo

Resumo: Observabilidade & Deploy

DECISÃO	IMPACTO
<code>console.log(event)</code> em prod	● Custo exponencial
Métricas sem dono	● Dinheiro invisível
Ambientes zumbis	● \$\$\$ rodando sem propósito
Build sem cache	● Tempo + custo desperdiçado
Retention + sampling + ephemeral	● Controle real

Autoscaling & Arquitetura

Patterns e anti-patterns que destroem orçamentos

Autoscaling sem limite máximo

```
# "Escala infinita" ...vai que né?  
AutoScalingGroup:  
  MinSize: 2  
  MaxSize: 999 # YOLO  
  DesiredCapacity: 2
```

O QUE ACONTECEU

- Pico de tráfego (ou bot)
- Escalou de 2 → **47 instâncias** em 10min
- Ninguém viu em tempo real

O CUSTO

~\$800
em 4 horas

Sempre defina **MaxSize** com base no **budget**, não no sonho.

Retry storms: quando o código DDoSa ele mesmo

```
Client → API → Lambda → SQS → Lambda → DynamoDB
      ↺ retry  ↺ retry  ↺ retry  ↺ retry
```

- 1 erro vira 1000 retries
- Cada retry custa compute, I/O e transfer

É tipo torcer para o CATS...você compra o ingresso e sabe que não vai ganhar!

Solução: Exponential backoff + circuit breaker + dead letter queue

Event-driven vs Polling

✗ POLLING (CRON A CADA 5S)

- 17.280 invocações/dia
- 90% retorna "nada pra fazer"
- Paga **24/7** pelo compute

✓ EVENT-DRIVEN (SQS + LAMBDA)

- Paga **por mensagem processada**
- Idle = \$0
- Escala com a demanda

Resumo: Autoscaling & Arquitetura

DECISÃO	IMPACTO
MaxSize: 999	● Fatura surpresa
Retry sem backoff	● DDoS em si mesmo
Polling pra tudo	● Custo constante desnecessário
Event-driven	● Paga pelo uso real
Circuit breaker	● Protege custo e sistema

Práticas & IA

O que fazer amanhã (literalmente)

5 práticas que custam \$0

1 Tags

Se não tem tag, não tem dono. Se não tem dono, ninguém corta

2 Lifecycle Rules

S3, logs, snapshots. Automação > memória humana

3 Right-size

AWS Compute Optimizer é grátis...Use e abuse!

4 Reserved / Savings Plans

Compromisso de 1 ano = -30~60%. Funciona pra compute, RDS, ElastiCache

5 Alarmes de billing

AWS Budgets com alerta...Zero surpresas!

IA como aliada de FinOps

O QUE A IA PODE FAZER

- Revisar arquitetura antes do deploy
- Identificar desperdícios em IaC
- Sugerir alternativas mais baratas
- Automatizar análise de custos

O SHIFT-LEFT DE FINOPS

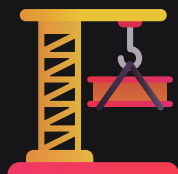
- Decisão de custo no **momento do código**
- Não na fatura 30 dias depois
- Review de PR com lente financeira
- "Quanto isso vai custar?" antes do merge (**Infracost** talvez?)

Não é substituir o engenheiro. É dar uma **lupa financeira** pro engenheiro.

Case: Kiro no fluxo de desenvolvimento

Dev: "Cria um módulo Terraform pra processar imagens"

Kiro: "Posso criar com Lambda, mas com 15min de execução e esse volume, ECS Fargate + Spot seria 90% mais barato. Quer que eu monte assim?"



Revisa IaC e aponta
custos potenciais



Sugere right-sizing
baseado no contexto



Identifica anti-patterns
antes do deploy

Kiro na prática

-  Identifica `log.debug` em prod, retry sem backoff, Multi-AZ em Dev
-  Propõe alternativas com estimativa de economia
-  Sugere Spot, Graviton, right-size com base no workload
-  Revisa Terraform/CDK com lente de custo

"É como ter um Tech Lead de FinOps no pair programming...que nunca reclama do café."

O Caminho

FinOps não é sobre cortar, é sobre decidir

FinOps ≠ "gasta menos"

FinOps = "gasta melhor"

✗ NÃO É

- Bloquear inovação por custo
- Microgerenciar centavos
- Criar burocracia pra provisionar
- Culpar o Dev pela fatura

✓ É

- Visibilidade pra quem decide
- Custo como **métrica de qualidade**
- Trade-offs conscientes
- **Eficiência sem trava**

Arquitetura boa é barata por design

- **Menos componentes**
= menos fatura
- **Menos dados desnecessários**
= menos storage
- **Menos chamadas inúteis**
= menos transfer
- **Menos over-engineering**
= menos compute

"Simplicidade não é preguiça. É senioridade."

O papel do Dev no ciclo de custo

Antes: Dev → Código → Deploy → ... → Fatura → 🚒 → "Ops, resolve"

Agora: Dev → Código → Review de custo → Deploy → Fatura → 😊

Shift-left de FinOps:

- Entender o pricing model ANTES de escolher o serviço
- Perguntar "quanto custa?" no design review
- Tratar budget como requirement, não como surpresa

O mínimo que todo Dev deveria ter acesso

FERRAMENTAS GRATUITAS

- **Cost Explorer:** Onde tá o dinheiro?
- **AWS Budgets:** Alarme quando passar de \$X
- **Tags:** Custo por time/projeto/feature
- **Trusted Advisor:** Quick wins de graça

"Se você nunca viu a fatura do seu projeto, você não sabe o custo das suas decisões."

Custo como métrica de qualidade

MÉTRICA TRADICIONAL	+ MÉTRICA DE CUSTO
Latência p99	Custo por request
Uptime 99.9%	Custo por usuário ativo
Deploy frequency	Custo por deploy
Test coverage	Custo dos ambientes de test

Se performance é KPI, custo também deveria ser...coloca ele no dashboard do time!

Bônus!

Porque deixar o mundo melhor para nossos filhos importa sim!

Green Code: eficiência que salva!

Cada CPU ociosa, cada loop mal escrito, cada query desnecessária, consome energia...e energia = carbono.

NA INFRA

- Ambientes efêmeros
- Regiões com energia renovável
- Graviton (ARM consome menos)

NO CÓDIGO

- Algoritmos eficientes ($O(n) > O(n^2)$)
- Batch em vez de loop de I/O
- Runtime/linguagem eficiente

Green Software Foundation: eficiência energética + carbon awareness + hardware eficiente. Custo baixo e pegada baixa andam *juntos*.

O Dev escreve carbono (sem saber)

1 Código eficiente = menos ciclos de CPU

Um algoritmo $O(n^2)$ que vira $O(n \log n)$ roda mais rápido, gasta menos compute e energia

2 Carbon-aware: rodar na hora certa

Batch jobs podem rodar quando/onde a rede elétrica está mais limpa (mais solar/eólica disponível)

3 Menos hardware desperdiçado

Serverless e containers bem dimensionados evitam servidores ociosos queimando energia à toa

A mesma otimização que **baixa sua fatura** também **baixa sua pegada de carbono**.

Duas vitórias, zero esforço extra.

Call to Action

Uma ação. Só uma....

Volte pro trabalho e olhe UMA fatura.

Só uma. Do seu projeto. Do seu time.

PERGUNTE

"O que tá rodando que não deveria?"

PERGUNTE

"O que tá no tamanho errado?"

PERGUNTE

"O que ninguém olha há meses?"

Uma pergunta já muda o jogo, e você não precisa virar FinOps engineer, precisa ter consciência!

Pra levar pra casa



Arquitetura = custo. Sempre.

Cada decisão técnica é uma decisão financeira.



Visibilidade primeiro, corte depois.

Não corte às cegas. Entenda onde o dinheiro vai.



Use IA para shift-left de custo.

Review de custo no momento do código, não na fatura.



Sem tag = sem dono = sem controle.

Tags são o GPS do seu dinheiro na cloud.



Custo é métrica de qualidade, não trava.

Eficiência e inovação andam juntas.

Referências e leituras recomendadas

- AWS Well-Architected: Cost Optimization Pillar
- AWS Compute Optimizer
- AWS Trusted Advisor
- AWS Pricing Calculator: calculator.aws
- Kiro: kiro.dev
- FinOps Foundation: finops.org
- Green Software Foundation: greensoftware.foundation

*"A Cloud é como **rodizio de pizzas**: você pode comer tudo, de todos os sabores, mas não significa que **deveria**!" 🍕*

HACK10WN
anos

